Understanding Retrieval Augmented Generation

RAG Architecture, Retrieval Techniques, Real-World Applications, and a Proposed Student Support System

Assignment: Understanding Retrieval Augmented Generation

Prepared on: 29 October 2026

Research report with proposed application and references

Abstract

RAG combines information retrieval with generative language models. Rather than relying only on information encoded during training, a RAG system searches a selected document collection and supplies relevant passages as context. This can improve grounding, support specialised knowledge, and make sources easier to inspect. This report explains RAG architecture, retrieval techniques, applications, limitations, safeguards, and a proposed student-support assistant.

1. Introduction

Large language models can generate fluent answers but may produce unsupported claims or lack access to specialised, current information. Retrieval-Augmented Generation addresses part of this problem by retrieving evidence at query time and placing it in the prompt. RAG does not guarantee correctness: the retriever can find poor evidence, and the model can misinterpret or ignore it.

2. RAG Architecture and Workflow

A typical pipeline has these stages: (1) ingest trusted documents; (2) extract and clean text; (3) split text into chunks with suitable overlap; (4) index chunks for search; (5) process the user's query; (6) retrieve relevant passages; (7) place the passages and question into a prompt; (8) generate an answer; (9) show citations and monitor quality. Permission checks should be applied before confidential passages are sent to a model. Source filenames, document versions, and timestamps improve traceability.

3. Data Retrieval Techniques

Keyword retrieval methods such as TF-IDF and BM25 rank documents using term overlap and term importance. They are fast and interpretable but may miss paraphrases. Dense vector retrieval represents queries and passages as embeddings and can match semantic similarity even when wording differs, but depends on embedding quality. Hybrid retrieval combines lexical and dense search. Re-ranking uses a second model to reorder candidates for relevance. Metadata filtering narrows results by attributes such as course, date, or user permissions. Chunk size and overlap should be tested on realistic questions: tiny chunks may lose context, while very large chunks may dilute relevance.

4. Importance and Benefits

RAG can connect a model to current documents without retraining the base model, support organisation-specific information, and make answers easier to verify through citations. Updating an index is often simpler than retraining a model. However, RAG does not eliminate hallucinations. Good source quality, relevant retrieval, grounded prompts, citations, evaluation, and a clear response when evidence is missing remain essential.

5. Real-World Applications

Customer support systems can retrieve approved manuals and troubleshooting articles before drafting answers. Enterprise knowledge assistants can search internal procedures, provided access controls prevent unauthorised disclosure. Educational assistants can explain course concepts using lecture notes and cite sources. Healthcare information tools may retrieve approved guidance for professional review, but RAG alone is not a clinical safety guarantee. Legal and policy research tools can locate passages in a defined corpus, while qualified professionals verify interpretation. These are application patterns; any company-specific case study should be supported by that company's published technical documentation or research.

6. Proposed Application: RAG Student Support Assistant

Students often need help with study methods, assignment planning, and digital learning. A general chatbot may invent college-specific policies. The proposed assistant searches approved student-support documents, retrieves relevant chunks, displays source filenames, and prepares a prompt that asks a model to answer only from the evidence and admit when information is missing. The included GitHub starter contains sample documents about assignment planning, digital learning, and study skills. It uses a simple lexical retrieval baseline and runs in demo mode without an API key. A live model requires a provider-specific SDK integration. A useful novel feature would be a source-coverage view that shows which parts of a question are supported and which remain unanswered, recommending a verified source instead of guessing.

7. Evaluation Plan

Evaluate retrieval and generation separately. Retrieval measures can include precision at k and recall against human-labelled questions. Answer evaluation should examine correctness, groundedness, completeness, citation accuracy, and whether the system appropriately declines to answer when evidence is absent. Operational measures include latency, cost, failure rate, and index-update time. Test common and paraphrased questions, ambiguous queries, outdated documents, and questions absent from the knowledge base. Security tests should check access permissions and prompt injection.

8. Challenges and Safeguards

Outdated documents can mislead users; record versions and assign source owners. Retrieval failure can be addressed through better chunking, hybrid search, and re-ranking. Hallucinations require citations, groundedness checks, and explicit uncertainty. Treat retrieved documents as untrusted data to reduce prompt-injection risk. Apply access controls before retrieval, minimise sensitive data, audit coverage for bias, monitor cost and latency, and keep human oversight for consequential decisions.

9. Future Improvements

The prototype could add semantic embeddings, vector search, hybrid retrieval, query rewriting, re-ranking, PDF ingestion with page references, multilingual queries, document versioning, feedback, and automated regression tests. A production system needs authenticated access, permission-aware retrieval, secure logging, monitoring, and an approval process for knowledge-base updates. Improvements should be measured against a baseline.

10. Conclusion

RAG combines retrieval with language-model generation to produce answers grounded in a selected document collection. Keyword, dense, hybrid, metadata-filtered, and re-ranked retrieval each offer different trade-offs. RAG improves traceability but does not remove the need for reliable sources, evaluation, privacy controls, and human oversight. The proposed student-support assistant demonstrates a practical starter workflow and highlights source coverage and explicit acknowledgement of unanswered questions as useful design goals.

References

[1] Lewis, P., et al. (2020). Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks. NeurIPS. https://arxiv.org/abs/2005.11401\n\n[2] Robertson, S., & Zaragoza, H. (2009). The Probabilistic Relevance Framework: BM25 and Beyond. Foundations and Trends in Information Retrieval, 3(4), 333–389. https://doi.org/10.1561/1500000019\n\n[3] Karpukhin, V., et al. (2020). Dense Passage Retrieval for Open-Domain Question Answering. EMNLP. https://arxiv.org/abs/2004.04906\n\n[4] NIST. (2024). Artificial Intelligence Risk Management Framework: Generative AI Profile (NIST AI 600-1). https://doi.org/10.6028/NIST.AI.600-1